Big Data Project — Adaptive Inference for Edge AI

Big Data at the Edge · Weeks 4–7 · 40% of final grade

1. Overview

Real edge deployments rarely see a single, fixed workload. Inputs vary in difficulty, sensors vary in availability, and the compute/energy/thermal budget available on a device can change from one moment to the next — a wearable on a low battery, a gateway under a burst of sensor traffic, an accelerator sharing capacity across models. A model that always runs at full size and full precision is leaving performance on the table whenever the input is easy or the budget is tight, and it simply fails when the budget can't cover its worst case.

This project asks you to design and build an adaptive inference system: a model (or family of models) that can change how much computation and how many resources it uses at runtime, and to characterize that system properly — not with a single accuracy number, but with a full accuracy-vs-cost trade-off curve or a *Pareto frontier*.

The approach is intentionally open-ended. You are given a toolbox of techniques from the course (Sections 2–4 below) and asked to figure out, and justify, which combination fits your chosen scenario. There is no reference implementation for this assignment — the design decisions are the assignment.

2. Learning objectives

- Combine compression techniques (pruning, quantization, knowledge distillation) with adaptive inference mechanisms (Bag of Models, early-exit, elastic/Slimmable) into a working system.
- Design and measure a genuine accuracy-vs-FLOPs (and latency/memory) Pareto frontier, using real measured FLOPs rather than theoretical alone.
- Justify architectural and algorithmic choices for a specific edge / big-sensor-data scenario, and defend those choices against alternatives.
- Critically evaluate whether an adaptive system actually outperforms a well-tuned static baseline — adaptivity is not free, and this project should teach you when it's worth its complexity.

3. The toolbox

The techniques below fall into two families. Compression techniques (3.1) reduce the cost of a single model. Adaptive inference mechanisms (3.2) let a system change how much of that cost it actually pays, per input or per resource condition, at runtime. A useful adaptive system usually needs at least one technique from each family — compression alone is static, and an adaptive mechanism with no compression behind it has nothing to trade off.

3.1 Compression techniques — “how to shrink a model”

Structured pruning. Removes whole structures — output channels, filters, attention heads — rather than individual weights, so the resulting tensors are physically smaller and inference is genuinely faster, not just sparser. Dependency-aware tools (e.g. DepGraph / torch-pruning, as used in Lab 2) are important here because removing a channel in one layer has knock-on effects on every layer it's connected to — including across skip connections. Structured pruning gives you a smaller, faster model, but on its own it's static: one pruning ratio, one fixed cost.

Quantization. Reduces the numerical precision used to store and compute with weights and activations (typically FP32 → INT8), shrinking memory footprint and, on suitable hardware, latency (Lab 3). Like pruning, standard post-training quantization produces one fixed operating point — useful as a multiplier on top of whatever adaptive mechanism you build, rather than a source of adaptivity by itself.

Knowledge distillation (KD). Trains a smaller "student" model to match the behaviour of a larger "teacher," transferring some of the teacher's accuracy into a cheaper model than training the student from labels alone would achieve. In an adaptive system, KD is often what makes aggressive compression (a very early exit, a heavily pruned sub-network, a high token-merging ratio) tolerable — the aggressively-compressed configuration is distilled from a stronger version of itself rather than trained in isolation.

3.2 Adaptive inference mechanisms — "how to make compute change at runtime"

Switchable models (Bag of Models). The system maintains multiple model variants — potentially different architectures, modalities, or compression levels — and switches between them depending on which data/task is currently available or what constraint is currently binding. Unlike early-exit's per-input granularity, switching is typically driven by a change in context: a sensor going offline, a task changing, an explicit resource budget changing. Switchable models can share weights (see the Nested / Shared Weights component in Section 6.3) or can be entirely separate models selected by a router.

Early-exit models. The network has multiple classification/output heads attached at different depths [4,5]. At inference time, an input can "exit" at an early head — skipping the remaining layers entirely — if that head is already confident enough, or continue to a deeper head if not. The compute spent on a given input is therefore a function of how easy that input is, not a fixed constant. The central design challenge is the exit criterion: a poorly calibrated confidence signal will exit confidently on wrong answers just as readily as on right ones.

Elastic / Slimmable supernet. A single network is trained so that many sub-networks — different widths, depths, or kernel sizes — can be "sliced" out of one shared set of weights and used directly, without retraining each configuration from scratch (e.g. Slimmable networks[1], Once-for-All-style[2] supernet, Elastoformer[3]). This produces an entire family of operating points from a single training run and a single deployed weight file, at the cost of a harder, more careful training procedure (naively training a shared-weight network across many widths tends to collapse accuracy at the extremes unless something like progressive shrinking or a "sandwich rule" is used).

---

4. The assignment

Choose (or propose your own) an edge / big-sensor-data inference scenario — see Section 7 for examples, including reusing the MobileNetV2 / CIFAR-10 pipeline from Labs 1–3 if you'd rather build on familiar code than adopt a new dataset. Design and implement an inference system for that scenario that produces not one model, but a family of operating points spanning a meaningful range of compute cost, and that can move between those operating points at runtime in response to either the input or an external resource signal.

How you get there is up to you. You might build an early-exit network whose exit-policy is set by a resource budget rather than the input; a distilled Slimmable supernet with a hand-designed width selection policy; or something that combines these differently. The one requirement is that your system's cost is not fixed — it must be capable of trading accuracy for compute (FLOPs) — and optionally latency or memory — at more than one point.

5. Minimum requirements

- A defined edge scenario, with a dataset and task appropriate to it (reusing/extending the Labs 1–3 setup is acceptable).
- A family of at least 3 distinct operating points spanning a meaningful compute range (e.g. via exit depths, pruning ratios, supernet slices, or merge ratios) — not just one compressed model.
- At least one compression technique (Section 3.1) and at least one adaptive inference mechanism (Section 3.2), used together.
- Real measured (or consistently calculated) FLOPs for every operating point, using the same methodology throughout (reuse/extend the benchmarking utilities from Labs 1–3); latency, memory, or parameter count may be reported as additional supporting evidence.
- A comparison against at least one static baseline (e.g. the original model, or a single statically pruned/quantized model) at matched accuracy or matched latency.

6. Scoring / KPIs

The primary artifact you are graded on is a Pareto frontier: a plot of accuracy (y-axis) against FLOPs (x-axis) across every operating point your system can produce, with your static baseline(s) plotted alongside for comparison. A good frontier is not just present — it should offer a trade-off against the static baseline (higher accuracy at matched compute, or lower compute at negligible accuracy loss) across a meaningful range, which is the evidence that your adaptive mechanism is buying something beyond what static compression alone would.

Latency and memory footprint (model size on disk, or peak runtime memory) are not separately scored, but reporting them per operating point alongside FLOPs is welcomed as optional supporting evidence within the Pareto Frontier component (Section 6.4) below — in many real edge scenarios, memory or latency is the binding constraint rather than raw compute, and including them strengthens your case that the trade-off matters for your chosen deployment.

Component	Weight	What we're looking for
Design of the Adaptive System	20%	Quality and clarity of the proposed adaptive system design — motivation, source of adaptivity, chosen mechanism, policy/controller design, and justification against the target edge deployment constraints (see Section 6.1).
Correct Adaptive Behaviour	20%	The implemented system genuinely adapts at runtime — a real, appropriate, evaluated change in computation/configuration/resource use in response to the adaptive signal, not just multiple independently-trained static models (see Section 6.2).
Nested / Shared Weights	15%	Award if the family of operating points shares weights through a single deployable artifact (e.g.

Component	Weight	What we're looking for
		Slimmable/elastic, nested configurations, a shared early-exit trunk, or nested pruning) rather than being independently-trained separate models (see Section 6.3).
Pareto Frontier — Accuracy vs. FLOPs	15%	A rigorous accuracy (y-axis) vs. FLOPs (x-axis) Pareto frontier across your operating points, using a consistent FLOPs methodology and compared against a static baseline, interpreted rather than just plotted (see Section 6.4).
Novelty, Use Case & Efficiency of Proposed Solution	10%	Meaningful additional value beyond a straightforward application of a course technique — e.g. novelty, a well-motivated use case, task/dataset difficulty, energy-efficient policy design, or resource-aware decision-making (see Section 6.5).
Presentation & Reflection	20%	Clear write-up and Week 7 presentation of design decisions, Pareto results, trade-offs, what didn't work, and reflective discussion connecting back to Big Data / edge constraints (see Section 6.6).

Total: 100%

6.1 Design of the Adaptive System — 20%

Assesses the quality and clarity of your proposed adaptive system design — the reasoning behind it, not just whether a technique has been implemented. Looks for:

- Motivation for the selected edge-AI scenario.
- The source of adaptivity you are responding to: input difficulty, resource availability, energy, thermal conditions, sensor availability, or another justified source.
- The adaptive solution selected — early exit, bag of models, elastic/slimmable networks, or a student-proposed approach.
- Policy/controller requirements and design, where applicable.
- Justification of your architectural and algorithmic choices.
- The connection between your design and the target edge deployment constraints.

6.2 Correct Adaptive Behaviour — 20%

Assesses whether the implemented system genuinely adapts at runtime, as distinct from simply training or deploying multiple independently trained static models. Looks for:

- Correct implementation of the adaptive mechanism.

- A runtime change in computation, model configuration, or resource use.
- An appropriate response to the selected adaptive signal.
- Evidence that the adaptive behaviour works as intended.
- Meaningful evaluation of the selected operating points.

6.3 Nested / Shared Weights — 15%

Assesses whether your operating points share model weights through a single deployable artifact, rather than being a set of independently-trained separate models. Examples include:

- Slimmable or elastic networks.
- Nested model configurations.
- Shared early-exit trunks.
- Nested pruned models, where smaller models use a strict subset of larger model weights.

Demonstrate or verify weight sharing where appropriate — no single architecture is mandated, provided your design satisfies the nested/shared-weight objective.

6.4 Pareto Frontier — Accuracy vs. FLOPs — 15%

The primary Pareto evaluation is accuracy vs. FLOPs. Requirements:

- Plot accuracy on the y-axis and FLOPs on the x-axis.
- Include the system's available operating points.
- Demonstrate a meaningful accuracy–compute trade-off.
- Compare against an appropriate static baseline where possible.
- Use a consistent FLOPs measurement or calculation methodology.
- Interpret the resulting Pareto frontier rather than merely presenting a plot.

Latency and memory are not required as primary axes and are not separately scored; they may be included as optional supporting evidence within this component when relevant to your chosen edge scenario, alongside energy or thermal measurements if useful. Optional evidence does not add to or alter the 100% base grading total.

6.5 Novelty, Use Case & Efficiency of Proposed Solution — 10%

Evaluates meaningful additional value in your proposed solution, beyond a straightforward application of a course technique. You do not need to demonstrate every dimension below — credit is earned through one or more well-justified contributions appropriate to your project, and unnecessary complexity for its own sake is not rewarded:

- Novelty or originality of the proposed approach.
- A novel or well-motivated edge-AI use case.
- Complexity or difficulty of the dataset/task.
- Energy-efficient policy design.
- Resource-aware or deployment-aware decision-making.
- A meaningful contribution beyond a straightforward application of a course technique.

6.6 Presentation & Reflection — 20%

Assesses, per the Week 7 presentation requirements:

- Clarity of the presentation and technical explanation.
- Motivation and scenario definition.
- Adaptive system design and policy.
- Experimental methodology and Pareto results.
- Discussion of trade-offs and limitations.
- What did not work and how you responded.
- Reflection on edge-AI and big-data constraints.
- Critical assessment of whether the proposed system meets its goals.

7. Suggested scenarios (optional inspiration)

You are not limited to this list, and you're encouraged to propose your own — but if you want a starting point:

- Wearable human activity recognition from IMU time-series, where battery level should influence how much compute is spent per inference.
- Always-on audio keyword spotting, where most windows contain silence/noise and should be handled by the cheapest possible path.
- Vision-based edge monitoring (traffic, industrial, environmental), where scene complexity varies a lot frame-to-frame.
- BERT-based natural language understanding, where the available compute budget, latency requirements, or input complexity influence the choice of model size or inference configuration for tasks such as sentiment analysis, intent classification, or question answering.
- Vision Transformer (ViT) for image classification at the edge, where input complexity, latency requirements, or battery constraints determine the appropriate model width, depth, or computational operating point.
- Vision at the edge, reusing the MobileNetV2 / CIFAR-10 pipeline from Labs 1–3 as a starting architecture and dataset, reframed around building a family of operating points rather than a single pruned/quantized model.

8. Deliverables

- Code (notebook(s) or repository) implementing your system, including the benchmarking used to produce your Pareto frontier.
- A short in-class presentation (Week 7).
- The presentation would include discussions on these items: scenario & motivation, toolbox choices & justification, system design, Pareto frontier (accuracy vs. FLOPs) results with baseline comparison, discussion of trade-offs and what didn't work, and the novelty, use case, or efficiency contribution of your proposed solution.

9. Tips

- Use DAS-6 for training or fine-tuning your models.
- Use CIFA-100/Tiny-Imagenet dataset instead of Imagenet (requires very long training), if using ViT for your project.
- Use all the previous Labs for more tips related to implementation and compression approaches.

- If the model is nested or shares weights, this can be verified by comparing the layer-by-layer weights in torch.
- Compute FLOPs with a consistent tool (e.g. fvcore, thop, or ptflops) across every operating point — mixing manual estimates with library-computed numbers will make your Pareto frontier hard to trust.
- Fix your static baseline before you start building the adaptive system, and keep it unchanged throughout — comparing against a moving target makes it hard to tell whether your adaptive mechanism is actually buying anything.
- If your adaptive mechanism relies on a confidence threshold or a learned controller, tune it on a held-out validation split, not on the data you'll report your final Pareto frontier from.

10. References

[1] Jiahui, Yu, et al. “Slimmable neural networks.” The International Conference on Learning Representations . 2019.

[2] Cai, Han, et al. "Once-for-all: Train one network and specialize it for efficient deployment." *arXiv preprint arXiv:1908.09791* (2019).

[3] Kalra, Sudaksh, and Dolly Sapra. "Elastoformer: Enabling Dynamic Adaptivity via Elastic Model Transformation." *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*. 2025.

[4] Teerapittayanon, Surat, Bradley McDanel, and Hsiang-Tsung Kung. "Branchynet: Fast inference via early exiting from deep neural networks." *2016 23rd international conference on pattern recognition (ICPR)*. IEEE, 2016.

[5] Zhou, Wangchunshu, et al. "Bert loses patience: Fast and robust inference with early exit." *Advances in Neural Information Processing Systems* 33 (2020): 18330-18341.